sentinel-oo: user manual

Anil Wipat, Newcastle University

11 November 2026

Contents

sentinel-oo: user manual **2**
What you will end up with . . . 2
Before you start . . . 3

Part 1: a tutorial **3**
The worked example . . . 3
Step 1: look at what you have been given . . . 3
Step 2: ask the question, then write one requirement . . . 3
Step 3: the trap, which you should walk into on purpose . . . 4
Step 4: tell the contract what it is for . . . 4
Step 5: build up the requirements about your own entities . . . 5
Step 6: a flag rather than a refusal . . . 6
Step 7: a value a model may be asked to recover . . . 6
Step 8: the second lesson . . . 8
Step 9: read your contract back in plain words . . . 9
Step 10: deploy it . . . 9

Part 2: the contract language **10**

Part 3: deploying **11**

Part 4: when something goes wrong **11**

Part 5: before you rely on it **12**

Writing a contract **13**
What a contract is . . . 13
The one question to ask about every requirement . . . 13
The shape of the file . . . 13
`expects`: what your contract is for . . . 13
Every field of a requirement . . . 14
`applies_to`: where it looks . . . 15
`property` or `named_value`: what it reads . . . 15
`must`: the five verbs . . . 15
`severity`: how hard it bites . . . 17
`one_of`: the only values that mean anything . . . 18
`repairable`: letting a model help . . . 18
What happens when you get it wrong . . . 18
What the language cannot say yet . . . 19
Checking your contract before you deploy . . . 19

Deploying your own instance **20**
One command . . . 20

What it does, in order . . . 20
What you see when it works . . . 20
The options . . . 21
Using it once it is running . . . 21
Two instances at once . . . 21
For a coordinator agent rather than a pipeline . . . 22
The model, and where it runs . . . 22
Who can reach it . . . 23
What your deployment folder needs . . . 23
If you prefer Docker Compose . . . 23
Stopping . . . 24

sentinel-oo: user manual

How to put a checking service at one of your own handovers. You will write contract files and run one command. You will not write any code.

This manual is in three files. This one is the front door and the tutorial. The other two are reference:

File	What it is for
This file	Start here. A tutorial that builds a contract from nothing
WRITING_A_CONTRACT.md	The contract language, field by field
DEPLOYING.md	The deployment program, option by option

If you want to know how the system is built rather than how to use it, read the system manual.

What you will end up with

A service running in a container that you send a handover to and get a verdict back from, plus the handover itself with a record of what was checked written inside it.

A **handover** is the moment one phase of work passes its output to the next. A **crate** is how that output travels: an RO-Crate, which is a folder of data files with a JSON file alongside them describing what each file is and how the things in it relate to one another.

The three verdicts

Verdict	What it means
`ACCEPTED`	Nothing was found
`ACCEPTED_WITH_FLAGS`	Nothing stops the handover, but something was recorded for somebody to look at. A value a model recovered from the crate's own text always lands here
`REFUSED`	At least one requirement with severity `refuse` was violated

What makes your instance yours

A folder of contract files, mounted into an image nobody modified. That is the whole of the tailoring.

Before you start

You need Docker, and a crate to try it on. If you have no crate of your own yet, use the worked example that ships with the product, which is what this tutorial does.

Everything below is run from the `sentinel-oo` folder.

Part 1: a tutorial

You can run every command here. The tutorial builds a contract one requirement at a time against the worked example that ships with the product, and the output shown is the output you will get.

The worked example

It is a handover from the Design phase to the Build phase of a bioprospecting cycle. Bioprospecting means searching genomes and metagenomes for enzymes that might be useful industrially.

The Design phase finishes with a panel of candidate proteins chosen from a metagenome, a synthetic gene designed for each one, and a plate layout saying which well each gene goes into. The Build phase orders those genes from a synthesis company and spends money, so anything Build cannot judge from the crate is a well that may be wasted.

Nothing in the service knows what a protein or a plate is. Every type name and value name below comes out of the contract file and the crate. The same image, unmodified, checks a coating line or a chemical plant.

All the sequence data in the example is simulated, and each sequence says so in its own line.

Step 1: look at what you have been given

```
python -m sentinel_oo --describe examples/plate-test/valid
```

It prints every entity type in the crate with a count, and the named values hanging off each. **That is the list your contract has to match**, because a requirement names a type and a value and both are compared as plain text.

That crate has 109 entities. The ones this tutorial uses are:

Entity type	What it is	How many
`PlateWell`	One numbered position on the plate	4
`ExpressionConstruct`	A synthetic gene designed for one protein	4
`Protein`	A candidate enzyme found in the metagenome	4
`IntendedPlateLayout`	The plate itself	1
`GeneCallingStep`	The step that predicted genes in the assembled DNA	1
`File`	A file the crate names, carried or elsewhere	11

Step 2: ask the question, then write one requirement

The question to ask about every requirement is: what does the receiving phase do, and what goes wrong if this is missing?

If a requirement has no answer to that, it is a preference rather than a requirement. Preferences accumulate until a contract is too irritating to use, and then people route around it, and then it protects nothing.

Here is one with an answer. If a well has no position, nobody can find it on the physical plate.

Make a folder and put this in it as `my_handover.yaml`:

```
contract: my_handover
version: 0.1.0

requires:
  - name: well_has_a_position
    applies_to: PlateWell
    property: name
    must: be recorded
    severity: refuse
    message: >-
      This well has no position, so nobody can find it on the physical plate.
```

Run it:

```
SENTINEL_CONTRACTS=<your folder> \
  PYTHONPATH=src python -m sentinel_oo examples/plate-test/valid \
  --contract my_handover

  109 entities in, 122 out
  verdict                                ACCEPTED
```

Every well has a position, so it passes. The crate goes in with 109 entities and comes out with 122, because the service wrote the record of this check into it.

Step 3: the trap, which you should walk into on purpose

Send it a crate from a completely different domain. One ships with the product for exactly this purpose: a library catalogue, which has no wells, no constructs and no gene calling step in it at all.

```
SENTINEL_CONTRACTS=<your folder> \
  PYTHONPATH=src python -m sentinel_oo examples/from-another-phase \
  --contract my_handover

  9 entities in, 22 out
  verdict                                ACCEPTED
```

It accepted a crate it knows nothing about. Your requirement is about `PlateWell`, that crate contains no `PlateWell`, so the requirement matched nothing and passed.

The requirement is behaving correctly. A rule about a type should say nothing about a crate with none of that type. The problem is the verdict drawn from its silence.

This is the worst thing the service can do, and it is worth understanding before you write anything else. Refusing something good is visible and sends somebody to look. Blessing something unchecked is silent and looks like success.

Step 4: tell the contract what it is for

Add an `expects` block listing the entity types your contract is written for:

```
contract: my_handover
version: 0.2.0

expects:
  - PlateWell
```

```
requires:
  - name: well_has_a_position
    ...
```

Now run both crates again. These are the same two commands as in steps 2 and 3:

```
SENTINEL_CONTRACTS=<your folder> \
  PYTHONPATH=src python -m sentinel_oo examples/plate-test/valid \
  --contract my_handover

SENTINEL_CONTRACTS=<your folder> \
  PYTHONPATH=src python -m sentinel_oo examples/from-another-phase \
  --contract my_handover

  the plate crate       ACCEPTED
  the other domain      REFUSED
                        REFUSE  nothing of type PlateWell is in this crate,
                                and the contract my_handover is written for
                                crates that contain it
```

Write this block in every contract you ever write.

Step 5: build up the requirements about your own entities

Raise the version number each time you change the contract, from `0.2.0` to `0.3.0` here, and once more at each of steps 6, 7 and 8. That is what makes the printout at step 9 read `version 0.6.0`. The service does not force this on you: it reads whatever number is in the file. It matters because the version is copied into the lasting record inside every crate the service checks, so a crate checked last month says which wording of the contract it was checked against.

Add the rest. Notice that references come in pairs: one requirement that the reference is recorded at all, and a second that it lands on something real. They are separate because one rule doing both would make one of them impossible to say.

```
expects:
  - PlateWell
  - ExpressionConstruct

requires:

  - name: well_has_a_position
    ...

  - name: one_well_per_position
    applies_to: PlateWell
    property: name
    must: be unique
    severity: refuse
    message: >-
      Two wells claim the same position on the plate, so what is physically
      in that position is undefined.

  - name: well_names_a_construct
    applies_to: PlateWell
    property: mainEntity
    must: be recorded
    severity: refuse
    message: >-
      This well does not say which construct is meant to go in it, so nothing
      measured from it later can be attributed to a sequence.
```

```
  - name: well_names_a_real_construct
    applies_to: PlateWell
    property: mainEntity
    must: point at something in the crate
    of_type: ExpressionConstruct
    severity: refuse
    message: >-
      This well names something that is not a construct in this crate, so
      whatever is in it cannot be identified.
```

Run the valid crate and two of the broken ones:

```
  valid                          ACCEPTED
  invalid/dangling-reference     ACCEPTED
  invalid/missing-conformsto     ACCEPTED
```

Two broken crates passed. Hold that thought until step 8.

Step 6: a flag rather than a refusal

Not everything worth saying should stop a handover. `severity: flag` records a finding and lets the crate through.

```
  - name: construct_length_is_plausible
    applies_to: ExpressionConstruct
    named_value: coding sequence length
    must: be a number between
    between: [300, 3000]
    severity: flag
    message: >-
      This construct's coding sequence length, in base pairs, is outside the
      range a single bacterial enzyme usually falls in, which is 300 to 3000
      base pairs.
```

`named_value` rather than `property`. A property sits directly on an entity. A named value is a separate entity carrying a name and a value, which the first one points at. Crates following the Process Run Crate profile record most numbers that way, and the four constructs here are 1053, 1080, 1101 and 1179 base pairs.

The valid crate still comes back `ACCEPTED`, because all four are in range. A crate with one construct of 41000 base pairs comes back like this:

```
  verdict                                ACCEPTED_WITH_FLAGS

  FLAG  This construct's coding sequence length, in base pairs, is outside
        the range a single bacterial enzyme usually falls in, which is 300
        to 3000 base pairs.
```

The handover is not stopped. Somebody is told.

Step 7: a value a model may be asked to recover

```
expects:
  - PlateWell
  - ExpressionConstruct
  - GeneCallingStep

requires:
  ...
```

```
- name: edge_handling_is_recorded
  applies_to: GeneCallingStep
  named_value: edge handling
  must: be recorded
  repairable: true
  severity: refuse
  message: >-
    The gene calling step does not say how it treated the edges of contigs,
    so nobody can tell whether any sequence in this panel may be a fragment.

- name: edge_handling_is_closed
  applies_to: GeneCallingStep
  named_value: edge handling
  must: be recorded
  one_of: ["closed"]
  severity: refuse
  message: >-
    This contract is written for a panel called with closed ends. A panel
    called any other way may contain fragments, and needs a contract that
    says what to do about them.
```

What edge handling is. Gene calling means running a program over assembled DNA to predict where the protein-coding genes are. When a predicted gene runs off the end of a DNA fragment, the program either keeps the partial gene or refuses to call it. `closed` means the second. It matters to the Build phase because a partial gene ordered as a construct makes a protein that is missing one end.

`repairable: true` means a model is asked one question when this fails: is the value already written down somewhere in the record, including inside free text? Everything it answers is checked by your rules before it counts, which is what makes having it on safe. A model runs in its own container and is called by default.

With no model reachable, which is what you get if you have not set one up:

```
valid                              ACCEPTED
invalid/missing-gene-calling-mode  REFUSED
invalid/edge-handling-in-prose     REFUSED
invalid/mode-in-prose              REFUSED
```

With a model reachable, one of those three changes:

```
invalid/edge-handling-in-prose     ACCEPTED_WITH_FLAGS

FLAG  The edge handling was not recorded on this entity. The value
      'closed' was recovered from the text already in the crate by
      qwen2.5:14b, and confirmed by the requirement
      edge_handling_is_recorded.
```

In that crate the gene calling step's description says, in ordinary words, "contig edges were treated as closed, so no partial genes were kept". The model was asked one question, read that sentence, proposed `closed`, and your own requirement checked the proposal before it was kept. On a laptop that run took about 9 seconds.

The other two do not change, and that is the point.

In `missing-gene-calling-mode` the value is not in the crate at all, so there is nothing to find and nothing is invented.

In `mode-in-prose` the value is in the crate, but only as the flag `-c` inside the command line `prodigal -i contigs.fasta -a proteins.faa -p meta -c -q`. Recovering it means knowing what that program's flags mean. Two models of 14 billion parameters each were asked to do exactly that on 10 November 2026, and both stated a wrong meaning without hedging. Your rules threw the answer out, the crate stayed refused, and nothing was written.

So the model helps when the value is in the text in words, and fails safely when it is not.

Step 8: the second lesson

Two broken crates were accepted back at step 5. **A contract catches only what it asks about**, and yours asks nothing about the packaging: whether the crate says which profile it follows, whether it lists files it never describes, or whether the files it carries hold anything.

Add those:

```
- name: declares_a_profile
  applies_to: root
  property: conformsTo
  must: be recorded
  severity: refuse
  message: >-
    The crate does not say which profile it follows, so nothing can be
    agreed about what its contents mean.

- name: everything_the_crate_lists_is_present
  applies_to: root
  property: hasPart
  must: point at something in the crate
  severity: refuse
  message: >-
    The crate lists a file as part of itself that is described nowhere in
    it. Whoever receives this will look for it and not find it.

- name: every_file_holds_data
  applies_to: File
  must: hold data
  severity: refuse
  message: >-
    This file is named in the crate and holds nothing, so the step that was
    meant to produce it produced no data.
```

`applies_to: root` means the entity describing the crate as a whole.

`hold data` opens the files themselves rather than reading the description. A file named by a web address is left alone, because the crate does not carry it.

With no model reachable, all six example crates now give this:

```
valid                                ACCEPTED
invalid/dangling-reference           REFUSED
invalid/missing-conformsto           REFUSED
invalid/missing-gene-calling-mode    REFUSED
invalid/mode-in-prose                REFUSED
invalid/edge-handling-in-prose       REFUSED
```

With a model reachable, `invalid/edge-handling-in-prose` is `ACCEPTED_WITH_FLAGS` instead, for the reason given in step 7.

And a refusal reads like this:

```
verdict                                REFUSED

REFUSE  The crate lists a file as part of itself that is described
        nowhere in it. Whoever receives this will look for it and not
        find it.
       at ./
```

Every sentence in that came from your contract file, not from the code.

Step 9: read your contract back in plain words

```
PYTHONPATH=src python -m sentinel_oo --show-contract my_handover

my_handover version 0.6.0

It is for crates containing all of: PlateWell, ExpressionConstruct,
GeneCallingStep
Each one that is missing is refused on its own, because a crate without it
cannot be checked by the requirements about it.

10 requirement(s):

  well_has_a_position
    PlateWell must record name
    failing it refuses the handover
    This well has no position, so nobody can find it on the physical plate.

  ...
```

This is what you send to the team that produces the handover, so they can check themselves before they are checked. It ends with a short explanation of each RO-Crate word it used, and it marks any requirement a model may help with.

Step 10: deploy it

```
python deploy.py <your folder of contracts>
```

That validates your contracts and refuses before building anything, builds the generic image, starts an instance with your folder mounted read only, and then confirms the running instance is serving exactly what it validated.

Then send it a crate. There are two ways, and they do different things.

The whole crate, zipped. This is the one to use. The service sends the checked crate back, which is the copy carrying the record of the check and the copy the next step in your workflow should receive. It is also the only way a requirement using the `hold data` verb can be answered, because that verb looks at the data files themselves.

```
cd <a crate> && zip -qr /tmp/crate.zip . && cd -

curl -s -D - -o /tmp/checked.zip \
  -X POST -H "Content-Type: application/zip" \
  --data-binary @/tmp/crate.zip \
  http://127.0.0.1:8080/check/my_handover
```

`-D -` prints the headers, which is where the verdict is, and `-o` saves the returned crate rather than printing a zip file into your terminal:

```
HTTP/1.1 200 OK
Content-Type: application/zip
Content-Disposition: attachment; filename="checked-ac7326ea4fb4.zip"
X-Sentinel-Run: ac7326ea4fb4
X-Sentinel-Verdict: REFUSED
X-Sentinel-Findings: 2
```

`HTTP/1.1 200 OK` means the service answered, not that the crate was accepted. The verdict is the `X-Sentinel-Verdict` line, and it is one of `ACCEPTED`, `ACCEPTED_WITH_FLAGS` or `REFUSED`. The reasons are inside the returned crate, in the record the service wrote into it.

The description file on its own. Quicker, and useful while you are still writing the contract, but nothing comes back except a report, and any requirement about the data files is refused because they were not sent.

```
curl -s -X POST --data-binary @<a crate>/ro-crate-metadata.json \
  http://127.0.0.1:8080/check/my_handover
```

That reply is JSON, and it is long: about 18 kilobytes for the small example crate, because it includes the whole checked description. Run as written it fills your terminal. Pick the verdict out of it instead:

```
curl -s -X POST --data-binary @<a crate>/ro-crate-metadata.json \
  http://127.0.0.1:8080/check/my_handover \
  | python -c "import json,sys
r = json.load(sys.stdin)
print(r['verdict'])
for f in r['findings']:
    print(' ', f['requirement'], '|', f['message'])"
```

For the `missing-gene-calling-mode` crate and the contract built in this tutorial, that prints (each sentence is one long line in the terminal; they are wrapped here to fit the page):

```
REFUSED
  edge_handling_is_recorded | The gene calling step does not say how it
      treated the edges of contigs, so nobody can tell whether any sequence
      in this panel may be a fragment.
  edge_handling_is_closed | This contract is written for a panel called
      with closed ends. A panel called any other way may contain fragments,
      and needs a contract that says what to do about them.
  every_file_holds_data | The data file 'panel.faa' was not sent, so
      whether it holds anything could not be checked, and a check that
      could not run is refused rather than passed. Send the whole crate as
      a zip rather than ro-crate-metadata.json on its own.
  every_file_holds_data | ... the same for constructs.json
  every_file_holds_data | ... and for plate-layout.csv
```

The last three are there because only the description file was posted. The three files the crate carries never arrived, so the requirement that they hold data could not be answered, and a check that could not run is refused rather than passed. Send the whole crate as a zip and those three go away. The eight files the crate names by web address are not mentioned at all, because the crate was never carrying them.

The reply has four fields: `run`, a twelve character identifier for this check; `verdict`; `findings`, one entry per requirement that failed, each naming the requirement, which entity failed it, and the contract author's sentence; and `crate`, the whole checked description.

Full details in DEPLOYING.md and API_MANUAL.md.

Part 2: the contract language

Every field, all five verbs, and the three things the language cannot say yet: WRITING_A_CONTRACT.md.

The five verbs are the whole vocabulary:

Verb	What it demands
`be recorded`	The value is present and not empty
`be unique`	No two entities of this type share it
`point at something in the crate`	The reference resolves to something described here
`be a number between`	It is a number, within a range
`hold data`	The file this entity describes is really in the crate and is not empty

You cannot add a sixth by writing a contract. If you need one, you will get a message naming the five that exist and saying where a sixth would be added, and you are blocked until somebody adds it to the product.

Part 3: deploying

DEPLOYING.md covers the deployment program, its options, running more than one instance, and the interface for a coordinating agent.

Part 4: when something goes wrong

What you see	What it means
`does not understand ['propery']`	A misspelt field. The message lists every field a requirement may have
`says 'be greater than', which is not something a requirement can demand`	A verb that does not exist. The message lists the four that do
`has no message`	Every requirement needs a sentence a person can read
`has severity 'warn'`	It is `refuse` or `flag`, and nothing else
`says 'be a number between' but does not say between`	That verb needs a `between: [low, high]`
`must say either property, for something written on the entity itself, or named_value`	A requirement reads one or the other, never both and never neither
`two files define the contract 'x'`	Two files in your folder declare the same contract name. The name inside the file is what matters, not the file name
`no contract called 'x'. This service knows: none at all.`	Nothing was mounted at `/contracts`, or the wrong folder was
`the running instance is not serving the contracts that were validated`	Usually the wrong folder being mounted. The message names what it expected and what it found

Everything is refused loudly rather than ignored. A contract containing something the reader does not understand is rejected whole, because a silently ignored requirement is a requirement you believe is protecting you.

It accepted something I expected it to refuse

Three usual causes:

Your contract does not ask about it. A contract catches only what it asks about. Step 8 of the tutorial is this mistake.

The type name does not match. Types are compared as plain text, and nothing knows that schema.org considers one type a kind of another. If your crate says `ExpressionConstruct` and your contract says `Construct`, the requirement matches nothing and passes.

You have no `expects` block. Run `--show-contract`. If it warns that the contract does not say what it is for, that is the cause.

Part 5: before you rely on it

A checklist. Each line is a mistake somebody has actually made.

- ☐ Every contract has an `expects` block listing the types it is for
- ☐ Every reference has a pair: one requirement that it is recorded, one that it lands on something real
- ☐ Every message is a sentence you would want to receive, and states units for every number
- ☐ Every requirement answers “what does the next phase do, and what goes wrong if this is missing?”
- ☐ You have run it against a crate you know is good **and** a copy you have deliberately broken
- ☐ `--show-contract` reads correctly to somebody who has never seen YAML
- ☐ You have sent that plain-words version to the team producing the handover
- ☐ `be unique` is only used where duplication really is wrong in your laboratory, not where it is how a replicate or a control is included

A contract that accepts everything is the same as no contract. The only way to know yours is doing something is to watch it refuse something.

Writing a contract

This is the whole of what you need in order to tailor `sentinel-oo` to your own handover. You will not write any code.

Read the worked example alongside this. It is 2 contract files and 5 crates, and it is short on purpose.

What a contract is

A handover is the moment one phase of work passes its output to the next. A contract is a file saying what must be true of that output before the next phase can act on it.

The service reads your contract, checks a crate against it, and returns one of three verdicts: ACCEPTED, ACCEPTED_WITH_FLAGS, or REFUSED. The middle one means nothing stopped the handover but something was recorded for a person or a later step to look at. A crate here means an RO-Crate: a folder of data files with a JSON file alongside them describing what each file is and how the things in it relate to each other.

What a contract is not. It is not a schema, because it does not say what shape the data must have. It is not a validator, because a third party validator already checks that the packaging is well formed. It is not a set of instructions to a model: there is no field for that, deliberately, and the reader refuses a file that invents one.

The one question to ask about every requirement

What does the receiving phase do, and what goes wrong if this is missing?

If a requirement has no answer to that, it is a preference rather than a requirement. Preferences accumulate until a contract is too irritating to use, and then people route around it, and then it protects nothing.

The shape of the file

```
contract: design_to_build
version: 0.11.0

description: >
  What Design must hand to Build, and why each item matters.

expects:
  - PlateWell
  - ExpressionConstruct

requires:
  - name: declares_a_profile
    applies_to: root
    property: conformsTo
    must: be recorded
    severity: refuse
    message: >-
      The crate does not say which profile it follows, so nothing can be
      agreed about what its contents mean.
```

Five top level keys and nothing else: `contract`, `version`, `description`, `expects`, `requires`. The name in `contract` is how callers ask for it, as in `POST /check/design_to_build`. The file name does not matter; this name does.

`expects`: what your contract is for

Write this block. A contract without one accepts crates it has not checked.

`expects` lists the entity types your contract is written for. If any of them is absent from the crate, the handover is refused before a single requirement is applied, and the message names the type that is missing.

```
expects:
  - PlateWell
  - ExpressionConstruct
  - IntendedPlateLayout
  - Protein
  - GeneCallingStep
```

Why it is not optional in practice

A requirement about a type says nothing about a crate containing none of that type, and it should not, because presence and soundness are separate things. Each requirement is behaving correctly. The problem is what happens when you add them up.

This was measured on 11 November 2026. A crate from an unrelated domain was posted to a contract of 18 requirements. The result was ACCEPTED with 0 findings:

Of the 18 requirements	Count
Looked at something in that crate	6
Matched no entity at all, and passed	12

The 12 were every requirement naming a type the crate did not contain. So the service said ACCEPTED on a crate it had not meaningfully read.

That is the worst thing this service can do. Refusing something good is visible and sends somebody to look. Blessing something unchecked is silent and looks like success. A handover from the wrong phase, or a crate in which one type name has been changed or misspelt, would come back accepted.

Why you choose the list rather than the service inferring it

The service could gather every type your requirements mention and demand all of them. It does not, because a contract may legitimately carry requirements about entity types that are optional, and only you know which of yours must be present. Inferring it would turn optional things into mandatory ones without asking.

So list the types whose absence means this crate is not the handover your contract is about.

Every field of a requirement

Field	Required?	What it does
`name`	Yes	How this requirement is referred to in a finding. Use words, not numbers
`applies_to`	Yes	Which entities it looks at
`property`	One of these two	A property written directly on the entity
`named_value`	One of these two	A value hanging off the entity as a separate entity
`must`	Yes	What it demands. One of exactly four things
`severity`	Yes	`refuse` or `flag`
`message`	Yes	The sentence a person reads when it is violated

Field	Required?	What it does
`one_of`	No	The only values that mean anything
`of_type`	No	Narrows a reference to one kind of thing
`between`	Only with the range verb	Two numbers, lowest first
`repairable`	No	Whether a model may be asked to recover the value

A requirement must say either `property` or `named_value`, never both and never neither.

`applies_to`: where it looks

Three possibilities.

`root` is the entity describing the crate as a whole. Use it for things true of the package: its name, its licence, which profile it follows, the list of files it claims to contain.

`descriptor` is the entity describing the metadata file itself. You will rarely want it.

Anything else is read as a type, so `applies_to: ExpressionConstruct` means every entity in the crate that says it is an `ExpressionConstruct`. You may write `type:ExpressionConstruct` if you want to be explicit.

A type here means either the entity's `@type` or its `additionalType`. The second matters in practice: RO-Crate's vocabulary has no term for a synthetic gene designed for expression, so the worked example types those entities as `CreativeWork` with `additionalType: ExpressionConstruct`, and a requirement can then name them.

A limitation worth knowing before you rely on it. Types are compared as plain text. Nothing in the service knows that schema.org considers a `Hotel` to be a kind of `LodgingBusiness`, so a requirement asking for one will not accept the other. If your crate uses a specific type and your contract names a general one, the requirement will match nothing and pass silently.

`property` or `named_value`: what it reads

`property` reads something written on the entity itself:

```
applies_to: PlateWell
property: name
```

`named_value` reads a value hanging off the entity as a separate entity of its own:

```
applies_to: ExpressionConstruct
named_value: coding sequence length
```

Both exist because crates following the Process Run Crate profile record most numbers and strings the second way. The coding sequence length is not written on the construct. It is a separate entity carrying the name `coding sequence length` and the value `1080`, which the construct points at.

You do not have to say which property carries the reference. Every reference the entity makes is followed, and only entities of type `PropertyValue` are read as values. So a step that points at a file, a tool and a setting will give up only the setting, and a file that happens to have a name is not mistaken for a value.

`must`: the five verbs

This is the whole vocabulary. It is deliberately short, and it grows one entry at a time, because a language that can say anything can also say things nobody can check.

You cannot add a verb by writing a contract. If your requirement needs one these five cannot express, you will get a message naming the five that exist, and you are blocked until a sixth is added to the product. That

means editing `MUSTS` in `src/sentinel_oo/contract.py`, adding a branch to `_apply` in `src/sentinel_oo/rules.py`, and writing tests for both. It is the one case where tailoring is not enough, and it is a change somebody has to agree to rather than something you can do in your own folder.

`be recorded`

The value must be present and not empty.

```
- name: well_has_a_position
  applies_to: PlateWell
  property: name
  must: be recorded
  severity: refuse
  message: >-
    This well has no position, so nobody can find it on the physical plate.
```

This is the only verb that fires when the value is absent. The others have nothing to say about something that is not there, because "it must be present" and "it must be sound" are two separate requirements. A rule doing both would make one of them impossible to say.

So presence and soundness come in pairs. Write both.

`be unique`

No two entities of this type may share this value.

```
- name: one_well_per_position
  applies_to: PlateWell
  property: name
  must: be unique
  severity: refuse
  message: >-
    Two wells claim the same position on the plate, so what is physically
    in that position is undefined.
```

Think about whether duplication is actually wrong in your laboratory. Two wells holding the same construct is a normal thing to do elsewhere: it is how a technical replicate or a positive control is included. A check that fires on correct practice is worse than no check. If replicates are normal for you, use `severity: flag` or leave the requirement out.

`point at something in the crate`

The reference must resolve to an entity the crate describes.

```
- name: well_names_a_real_construct
  applies_to: PlateWell
  property: mainEntity
  must: point at something in the crate
  of_type: ExpressionConstruct
  severity: refuse
  message: >-
    This well names something that is not a construct in this crate, so
    whatever is in it cannot be identified.
```

With `of_type`, the thing it lands on must also be of that type. Without it, anything in the crate will do.

A reference that points outside the crate, to a URL somewhere, counts as not resolving. That is the point: a crate that refers to things it does not carry or describe cannot be acted on by somebody who only has the crate.

`be a number between`

The value must be a number, and within the range.

```
- name: construct_length_is_plausible
  applies_to: ExpressionConstruct
  named_value: coding sequence length
  must: be a number between
  between: [300, 3000]
  severity: flag
  message: >-
    This construct's coding sequence length, in base pairs, is outside the
    range a single bacterial enzyme usually falls in, which is 300 to 3000
    base pairs.
```

`between` takes two numbers, lowest first, and the range includes both ends.

State the units in the message every time. The contract has no field for units, and a number with no unit in a finding is an invitation to compare a length in base pairs against a concentration in nanograms per microlitre.

`hold data`

The file this entity stands for exists and is not empty.

```
- name: every_file_holds_data
  applies_to: File
  must: hold data
  severity: refuse
  message: >-
    This file is named in the crate and holds nothing, so the step that
    was meant to produce it produced no data.
```

Every other verb reads the description. A description can be perfectly correct about a file that is empty, which is what happens when a step runs, writes nothing and exits successfully. Nothing else here would notice.

It takes no `property` and no `named_value`, because it is about the file an entity stands for rather than anything written on it.

It says nothing about whether the data is right. That is a different question and not this service's to answer.

The data has to be there. A crate sent as a bare `ro-crate-metadata.json` is its description with the data left behind, and this refuses rather than passing in that case, because a check that could not run must not look like a check that passed. Send a zipped crate, or give the service a folder.

A file identified by a URL is not looked at. The crate does not carry it, and fetching it is a different act with different consequences.

`severity`: how hard it bites

Two values and nothing in between.

`refuse` stops the handover. The verdict becomes REFUSED and whatever comes next should not run.

`flag` records the finding and lets the handover through. The verdict stays ACCEPTED and the finding is written into the returned crate, where something downstream can act on it or a person can read it.

There is no middle value on purpose. A severity nobody can act on is a severity nobody reads.

Use `flag` when the answer is a judgment for a person, and `refuse` when proceeding would waste money or produce a record that is wrong.

`one_of`: the only values that mean anything

```
- name: edge_handling_is_closed
  applies_to: GeneCallingStep
  named_value: edge handling
  must: be recorded
  one_of: ["closed"]
  severity: refuse
  message: >-
    This contract is written for a panel called with closed ends, meaning
    that a predicted gene running off the end of a contig was discarded
    rather than reported truncated. A panel called any other way may
    contain fragments, and needs a contract that says what to do about
    them.
```

A known awkwardness, stated so it does not surprise you. `one_of` hangs off `be recorded`, so when the value is absent both the presence requirement and this one fire, and the report says two things rather than one. Both are true, but it is noise. There is no way to say "if present, must be one of these" yet.

`repairable`: letting a model help

```
repairable: true
```

Marking a requirement repairable means a model may be asked one question when that requirement fails: **is this value already written down somewhere in the record, including inside free text?**

That is the whole of what a model does here. It is the one job rules cannot do, because a rule can only read a value recorded as a value, and a value sitting inside a sentence is invisible to it.

Everything a model proposes is then checked by your rules before it counts, and it is kept only if every requirement reading that value now passes and nothing new is broken anywhere. The returned crate records that the value was recovered from text rather than stated, and which engine proposed it.

A value a model recovers raises a flag, naming the value, the model that proposed it and the requirement that confirmed it. So a crate rescued this way comes back as `ACCEPTED_WITH_FLAGS` rather than `ACCEPTED`. The handover is not blocked and nobody has to act. The flag is there because the crate itself already says a value read out of prose is less trustworthy than one somebody wrote down, and the verdict should say the same thing rather than contradict it. The person worth telling is the team producing the handover: a step that relies on a model to recover what it should be recording is a fault further up the workflow.

Repairable is not the same as guessable. Nothing may be invented. The value either is in the record or it is not, and answering that it is not there is a correct and expected answer.

What this was measured to do. On 10 November 2026 two models running locally, each with 14 billion parameters, were asked to do exactly this job on a real crate. Neither could. Every outcome was a refusal with nothing written, which is the system behaving correctly. A model being wrong, slow, absent or confidently mistaken changes no verdict. Mark a requirement repairable because it would be useful if it worked, not because you are relying on it.

A model is called by default, so `repairable: true` takes effect as soon as a contract is deployed. A deployment turns it off by setting the model address to nothing, or a caller by passing a repair budget of 0.

What happens when you get it wrong

The reader refuses the whole contract and names the fault. It does not ignore what it does not understand, because a silently ignored requirement is a requirement you believe is protecting you.

```
trial.yaml: requirement 1 (r) says 'be greater than', which is not something
a requirement can demand. It may say: ['be a number between', 'be recorded',
'be unique', 'hold data', 'point at something in the crate'].
```

```
trial.yaml: requirement 1 does not understand ['propery']. A requirement may
say ['applies_to', 'between', 'message', 'must', 'name', 'named_value',
'of_type', 'one_of', 'property', 'repairable', 'severity'] and nothing else.
In particular there is no field for instructing a model.

trial.yaml: requirement 1 (r) has no message. A violation nobody can read is
not worth raising.

trial.yaml: requirement 1 (r) has severity 'warn'. It is ['flag', 'refuse']:
either the handover stops or it is mentioned, and there is nothing in between
that anybody can act on.
```

What the language cannot say yet

Three gaps, recorded rather than worked around. If you need one of these, you are blocked until a verb is added to the product, and that is a change to the code rather than to your file.

It cannot compare one value against another. Every requirement reads one value and judges it on its own.

Here is a real example of what that costs. Each construct records its coding sequence length in base pairs, and the protein it was designed from records its amino acid sequence. The length should be three times the number of amino acids, plus three for the stop codon. That holds exactly for all four constructs in the worked example: 359 amino acids and 1080 base pairs, 366 and 1101, 350 and 1053, 392 and 1179. A construct built from the wrong protein would break it. No requirement you can write will notice, for two reasons: nothing can compare two values, and nothing can measure the length of a sequence, since the crate records the sequence itself and not its length.

There are no conditional requirements. You cannot say that if the gene calling step was run in metagenomic mode then the sample's environment must also be recorded. Each requirement fires or does not fire regardless of anything else in the crate.

There is no way to say "if present, must be one of these". `one_of` hangs off `be recorded`, so a missing value triggers both requirements.

Checking your contract before you deploy

The service prints a contract back in plain words, which is what a producing team reads to check themselves before they are checked:

```
python -m sentinel_oo --show-contract design_to_build
```

Then run it against a crate you know is good, and against a copy you have deliberately broken. A contract that accepts everything is the same as no contract, and the only way to know yours is doing something is to watch it refuse something.

See DEPLOYING.md for getting it running.

Deploying your own instance

You need a folder of contract files. That is the whole of what makes an instance yours, and you will not write any code.

See WRITING_A_CONTRACT.md for the contract language.

One command

```
python deploy.py <your folder of contracts>
```

For example, using the worked example that ships with the product:

```
python deploy.py examples/contracts
```

The deployment is a program rather than a page of instructions on purpose. Nothing is tailored by hand and no person follows steps.

What it does, in order

It stops at the first thing that fails, and the exit code is 1 when it stops.

1. It reads every contract in your folder and refuses the deployment if any of them is malformed, naming the file, the requirement and what a requirement may say.

This happens before anything is built or started. A deployment that starts with a broken contract discovers the problem at first use, which means discovering it in front of whoever sent the handover. Finding out here costs nobody anything.

It also refuses a folder that holds no contract files at all, because an instance with none would refuse every handover it was given.

2. It builds the generic image from the product folder. The product's own test suite, 200 tests, runs inside that build, and the image cannot be produced if any of them fails.

Pass `--no-build` to use an image that already exists.

3. It starts an instance with your contracts folder mounted read only at `/contracts`. The image is not modified and holds no contracts of its own.

Read only is deliberate: a service that checks handovers has no business editing the contract it checks against.

4. It checks the running instance is serving what was validated, comparing the contract names, their versions, and a checksum over the files.

That catches two things. The wrong folder being mounted, which is the common mistake. And a contract edited between step 1 and step 3, which is a real window because the build takes about a minute.

What you see when it works

```
sentinel-oo: deploying a tailored instance

1. Reading contracts from examples/contracts
   build_to_test               0.3.0      2 requirements
   design_to_build             0.11.0    20 requirements
   checksum over all of them: sha256:b26c400c...

2. Building the generic image
   sentinel-oo:dev built
   the product's test suite ran inside the build and passed

3. Starting the instance
```

```
   container sentinel-examples started from sentinel-oo:dev
   contracts mounted read only from examples/contracts

4. Checking the instance serves what was validated
   answering on http://127.0.0.1:8080
   image dev, commit c0525b0
   model: reachable  (qwen2.5:14b at http://model:11434)
   2 contract(s) served, checksum matches what was validated

Deployed.
```

The options

Option	What it does
`--check-only`	Validate the contracts and stop. Nothing is built or started
`--no-build`	Use the existing image instead of building
`--crates <folder>`	Mount a folder of crates read only at `/crates`, for trying it
`--port <n>`	Which port to serve on. The default is 8080
`--name <n>`	The container name. The default is `sentinel-<your folder's parent>`
`--image <n>`	The image to build and run. The default is `sentinel-oo:dev`
`--model-url <url>`	Where the model is. An empty value means no model
`--model <name>`	Which model to use, pinned by name
`--reachable-from-network`	Let other machines reach it. Off by default
`--yes`	Agree in advance to downloading the model
`--stop`	Stop and remove the instance, the model beside it, and the network between them

Using it once it is running

```
curl -s http://127.0.0.1:8080/contracts

curl -s -X POST --data-binary @<a crate>/ro-crate-metadata.json \
  http://127.0.0.1:8080/check/<your contract name>
```

The reply carries the verdict, the findings, and the crate with the service's record of what it checked written inside it.

Method and path	What it does
`POST /check/{contract}`	The whole job. A crate in, a checked crate out
`GET /contracts`	Which contracts this instance holds, with versions and a checksum
`GET /contracts/{name}`	One contract written out in plain words
`GET /runs`	The runs this instance is still holding
`GET /health`	The build it is running and which model, if any
`GET /openapi.json`	A description of the service, generated from its route table

Two instances at once

The image is generic, so one image serves any number of deployments. Give each one its own port and name:

```
python deploy.py <your own contracts> --port 8080
python deploy.py examples/contracts --port 8081 --name sentinel-example
```

That was run on 11 November 2026 and both instances served at the same time from one unmodified image, each holding only its own contracts and each refusing a contract name the other one knew.

For a coordinator agent rather than a pipeline

The same image speaks the Model Context Protocol on standard input and output instead of HTTP. An agent starts it when it wants it:

```
docker run -i --rm \
  -v "$PWD/<your contracts>:/contracts:ro" \
  sentinel-oo:dev --agent-interface
```

It offers four tools: `check_handover`, `list_handovers`, `read_contract` and `get_run_graph`. It never returns the crate, only the verdict, the findings and the path the crate was written to, because a crate is thousands of lines of JSON and putting that in an agent's context costs money on every call and invites the agent to start editing laboratory records.

A test fails if the two interfaces ever give different verdicts.

Not yet tested against a real Model Context Protocol client. It is tested against the protocol as written down.

The model, and where it runs

A model is called by default. A sentinel at a handover without one is not what this project is about.

It runs in its own container, which does not have to be on this machine. A model worth using wants a GPU, and the machine with the GPU is usually not the machine running the service.

Option	Environment variable	What it is
`--model-url`	`SENTINEL_MODEL_URL`	Where the model is. Default `http://model:11434`
`--model`	`SENTINEL_MODEL`	Which model, pinned by name. Default `qwen2.5:14b`
	`SENTINEL_MODEL_FIRST_TIMEOUT`	Seconds to wait for the first answer. Default 180
	`SENTINEL_MODEL_TIMEOUT`	Seconds to wait for each answer after that. Default 60

```
python deploy.py <contracts>                                a model beside it
python deploy.py <contracts> --model-url http://gpu:11434   a model elsewhere
python deploy.py <contracts> --model-url ""                 no model at all
```

`compose.yaml` starts a model container alongside the service, using the `ollama/ollama` image, with a volume so the model is not downloaded again on every start. On a machine with an NVIDIA GPU, uncomment the `deploy` block in that file to hand the GPU to the model container.

Knowing whether it is actually there

`deploy.py` says so at step 4, and `GET /health` reports one of three states:

State	Meaning
`none configured`	No model was asked for. Deliberate

State	Meaning
`reachable`	A model is configured and answering
`configured but not answering`	A model was wanted and cannot be reached

The third carries the error and says what follows from it: checks run normally, no verdict changes, and a value recorded only in free text cannot be recovered. Before 11 November 2026 all three looked identical, which is how an instance can appear to be working while doing none of what it was deployed to do.

Why having it on is safe

Not because models are reliable. Because of where it sits.

It is asked one question, about a check that has already failed, at a place the rules named. Whatever it answers is written to a copy and the whole contract is run again, and the change is kept only if every requirement reading that value now passes and nothing new broke anywhere.

Measured on 11 November 2026 against a running instance: given a crate whose value sits inside a command line, a 14 billion parameter model recovered it and the handover was accepted in about half a second. Given a crate where the value is not present at all, it wrote nothing and the handover was refused.

Who can reach it

By default, only this machine. The port is published on the loopback interface, so other machines on your network cannot send it requests.

The service asks for no password of any kind. That is why the default is what it is. Before 11 November 2026 it published on every interface, which on shared wifi at a conference means anyone on that network could use it.

To let other machines reach it:

```
python deploy.py <contracts> --reachable-from-network
```

It says so when it starts, in those words, because somebody doing it by accident should find out immediately. Put it behind something that authenticates before you use it anywhere you do not control.

What your deployment folder needs

Only the contracts. Everything else is optional:

```
your-deployment/
  contracts/
    your_handover.yaml        the whole of the tailoring
  crates/                     optional, for trying it
  README.md                   optional, what this deployment is for
```

There is a worked one in `examples/`.

If you prefer Docker Compose

`deploy.py` uses `docker run` directly, so a deployment needs no compose file at all. `compose.yaml` in this folder does the same thing for somebody who would rather use Compose:

```
SENTINEL_CONTRACTS=../my-deployment/contracts docker compose up
```

Setting	What it does	Default
SENTINEL_CONTRACTS	Your folder of contracts	The worked example
SENTINEL_PORT	Which port	8080, which collides with deploy.py
SENTINEL_BIND	Which interface	127.0.0.1, this machine only
SENTINEL_MODEL_URL	Where the model is	A container beside it
SENTINEL_MODEL_FIRST_TIMEOUT	Seconds to wait for the first answer	180
SENTINEL_MODEL_TIMEOUT	Seconds to wait for later answers	60

Three things it does not do, which is why deploy.py is the ordinary route: it does not check your contracts before starting, it does not tell you that it is about to download about 11 gigabytes and wait for a yes, and it does not confirm afterwards that the running instance serves what you meant.

Checked crates go into a named volume rather than a folder, so nothing is written into the repository. Copy them out with docker compose cp sentinel:/out ./out.

Stopping

```
python deploy.py <your folder of contracts> --stop
```